LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05 – RELASI ANTAR KELAS



NAMA: Nur Tsalits Alwan Mubarok

NIM: 103132400039

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam pemrograman berorientasi objek, sebuah sistem biasanya terdiri dari banyak kelas yang saling berhubungan. Hubungan antar kelas ini dimodelkan dengan diagram kelas, yaitu diagram yang menggambarkan kelas beserta atribut, method, dan relasinya. Jenis relasi yang dibahas pada modul ini adalah asosiasi, agregasi, dan komposisi.

1) Asosiasi adalah hubungan antara dua kelas terpisah yang saling berinteraksi melalui objek, tanpa salah satunya menyimpan objek lain sebagai atribut permanen. Contohnya kelas Dosen yang memberi nilai kepada objek Mahasiswa yang diterima lewat parameter method.
2) Agregasi adalah hubungan bagian dan keseluruhan yang lebih erat, tetapi bagiannya tetap dapat dibuat dan hidup sendiri. Contohnya kelas Jurusan yang menampung kumpulan objek Mahasiswa.
3) Komposisi adalah bentuk khusus dari agregasi dengan hubungan yang lebih kuat. Pada komposisi, bagian tidak dapat berdiri sendiri tanpa kelas utamanya, sehingga objek bagian dibuat di dalam kelas utama dan ikut hilang ketika objek utamanya hilang. Contohnya kelas Komputer yang memiliki Processor, Monitor, dan Mouse.

B. Guided

1. Asosiasi

```
//file Mahasiswa.java

package guided;

public class Mahasiswa {

    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }
```

```
    public int getNilai(){
        return this.nilai;
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }
}
```

```
//file Dosen.java

package guided;

public class Dosen {

    private String kodeDosen;
    private String namaDosen;
//Setter

    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }
//Getter

    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        // ini asosiasi, method milik class Student dipanggil di class Dosen,
// tp objek Student tidak menjadi atribut dr class Lecture
s.setNilai(nilai);
```

```
    }

    public int getScore(Mahasiswa s) {
        // ini asosiasi, method milik class Student dipanggil di class Lecture

// tp objek Student tidak menjadi atribut dr class Lecture

    return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");
        Dosen d = new Dosen();
        d.giveScore(m, 90);
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output:

```
Nim :130118383
Nama :Budi
Nilai :90
```

2. Agregasi

```
//file Mahasiswa.java

package guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
```

```
    }

    public String getNama(){
        return Nama;
    }
    //Setter
    public void setNim(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

```
//file Jurusan.java

package guided.agregasi;

public class Jurusan {
    private int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //behaviour
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa]= m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: "+ this.kodeJurusan);
        System.out.println("Nama Jurusan: "+ this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for(int i = 0; i<jumlahMahasiswa;i++){
        System.out.println(i+1 + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
```

```
    }
  }

    //getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    //setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

```
//file main.java
import guided.agregasi.Jurusan;
import guided.agregasi.Mahasiswa;

public class Project_Modul_5 {

    public static void main(String[] args) {
        runGuidedAgregasi();
    }

    public static void runGuidedAgregasi() {
        Mahasiswa m1 = new Mahasiswa("103112400148", "Messi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "Yamal");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }
}
```

Output:

```
=== GUIDED AGREGASI ===
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1. 103112400148 Messi
2. 103112400149 Yamal
```

3. Komposisi

```
//file Processor.java

package guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
```

```
//file Monitor.java

package guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

```
//file Mouse.java
package guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```
//file Komputer.java

package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String
brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

```
//file Main.java

package guided.komposisi;

public class Main {
```

```
    public static void main(String[] args) {
        Komputer k = new Komputer("Intel", 3, "LG", "Logitech", "Wireless");
        k.displaySpecKomputer();
    }
}
```

Output:

```
=== SPESIFIKASI KOMPUTER ===
Intel, 3
LG
Logitech, Wireless
```

C. Unguided

1) Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe. Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

1) Tugas 1: Implementasi Komposisi

   Buatlah dua class: RincianHarga dan ItemPesanan.

2) Tugas 2: Implementasi Agregasi

   Buatlah class Pesanan

3) Tugas 3: Implementasi Asosiasi

   Buatlah class Kasir

4) Tugas 4: Eksekusi dan pengujian Main

   Bualah class main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

Jawaban:

```
//file RincianHarga.java

package unguided;

public class RincianHarga {

    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
```

```
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

```
//file ItemPesanan.java

package unguided;

public class ItemPesanan {

    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // Komposisi: RincianHarga dibuat di dalam constructor ItemPesanan
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

```
//file Pesanan.java

package unguided;

public class Pesanan {
```

```
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    // Agregasi: item dibuat di luar, lalu dimasukkan lewat method ini
    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, item " + item.getNamaItem() + " tidak dapat
ditambahkan.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            RincianHarga r = this.daftarItem[i].getRincian();
            System.out.println((i + 1) + ". " + this.daftarItem[i].getNamaItem()
                    + " - Rp " + r.getHargaTotal()
                    + " (Harga: " + r.getHargaDasar()
                    + ", Pajak: " + r.getPajak() + ")");
        }
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
```

```
    }
}
```

```
//file Kasir.java

package unguided;

public class Kasir {

    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    // Asosiasi: Pesanan hanya diterima lewat parameter, tidak disimpan sebagai field
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = pesanan.getDaftarItem()[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }
        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

```
//file Main.java

package unguided;

public class Main {

    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");
```

```
        // Agregasi: ItemPesanan dibuat independen di main
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);

        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();
        System.out.println();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output:

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
============================
```

Penjelasan:

Program ini terdiri dari lima kelas yang menerapkan tiga jenis relasi.

1) Komposisi

   Kelas RincianHarga menyimpan hargaDasar dan pajak. Pajak dihitung otomatis sebesar 10% dari harga dasar di dalam constructor-nya, lalu getHargaTotal() mengembalikan jumlah keduanya. Kelas ItemPesanan membuat objek RincianHarga langsung di dalam constructor-nya menggunakan new. Karena objek RincianHarga dibuat dan dimiliki oleh ItemPesanan, keduanya tidak dapat dipisahkan, inilah yang membuatnya disebut komposisi.

2) Agregasi
   Kelas Pesanan menyimpan ItemPesanan dalam array daftarItem, dengan jumlahItem sebagai penanda posisi slot yang sudah terisi. Method tambahItem() memasukkan item ke array dan memeriksa kapasitas agar tidak melebihi ukuran array. Objek ItemPesanan dibuat lebih dulu di main, baru dimasukkan ke Pesanan, sehingga item tetap bisa ada walaupun Pesanan dihapus. Method tampilkanPesanan() mencetak nomor meja dan setiap item beserta harga total, harga dasar, dan pajaknya.
3) Asosiasi
   Kelas Kasir hanya memiliki field namaKasir dan tidak menyimpan objek Pesanan. Objek Pesanan diterima lewat parameter method prosesPembayaran(). Di dalamnya, program melakukan perulangan untuk menjumlahkan harga dasar dan pajak seluruh item, lalu mencetak struk berisi subtotal, pajak, dan total bayar. Hubungan Kasir dan Pesanan hanya terjadi saat method dipanggil.
4) Main
   Kelas Main membuat satu Kasir, tiga ItemPesanan, dan satu Pesanan untuk Meja 4. Ketiga item dimasukkan ke pesanan, pesanan ditampilkan, lalu Kasir memproses pembayaran. Hasilnya total bayar Rp 71500.0, terdiri dari subtotal Rp 65000.0 dan pajak Rp 6500.0.

Kesimpulan

Dari praktikum Modul 5 dapat disimpulkan bahwa asosiasi, agregasi, dan komposisiadalah tiga bentuk hubungan antar kelas yang berbeda dalam keeratan dan siklus hidup objeknya. Asosiasi hanya berupa interaksi sesaat melalui parameter method, agregasi menyimpan objek yang dibuat di luar dan tetap bisa berdiri sendiri, sedangkan komposisi membuat objek bagian di dalam kelas utamanya sehingga keduanya terikat.

Ketiga relasi tersebut dapat dikenali dari cara penerapannya di kode. Pada komposisi objek dibuat dengan new di dalam constructor, pada agregasi objek dibuat di luar lalu disimpan, dan pada asosiasi objek hanya diterima lewat parameter tanpa disimpan sebagai field. Dalam praktikum ini juga terlihat bahwa kesalahan kecil seperti salah penulisan tipe data, nama variabel, atau penggunaan static yang tidak tepat dapat menyebabkan program gagal dijalankan atau menghasilkan data yang tercampur.